Let's take a closer look at how this class works:

1. On line 4, we register the formatter with RSpec, passing it a list of events we want notifications about. Here, we only care about the :example_failed event. To see what other event notifications are available to formatters, check the API docs.[4]

2. In the formatter's initializer on line 6, we store the output argument RSpec passes us, so that we'll know where to send error messages. This object is a standard Ruby IO instance: either the standard output stream or a file on disk. We also set up an index to track which failure we last saw—we'll need this later to number the failures in the output.

3. Line 11 is the heart of our formatter: the example_failed callback we registered earlier. RSpec will pass us a 'notification' object containing details about the event. This object also has useful formatting helpers, such as the fully_formatted method that returns the output for a specific failure.

We've now seen how the formatter works once it's running, but we haven't talked about how it gets configured to run in the first place. Let's do so now.

Getting RSpec to Use the Formatter

Users of the new formatter are going to launch RSpec with the --require 'rspec/print_failures_eagerly' option. That flag will load the PrintFailuresEagerly::Formatter class into memory, but some piece of code will need to configure RSpec to use this class as a formatter.

For simpler formatters, RSpec provides a configuration API called add_formatter. If you were using this API, you'd call the helper inside a standard RSpec.configure block like so:

09-configuring-rspec/02/configuring_rspec/spec/spec_helper.rb

```
RSpec.configure do |config|
  config.add_formatter MyFormatter
end
```

However, it will take a little more finesse to configure this formatter. The code we just looked at will print only failure messages. But we also want to see whatever other output would normally appear, such as progress dots or example descriptions.

We'll need another formatter to supply the bulk of the output. We could just rely on users to pass a formatter explicitly to RSpec via the command line,

4. http://rspec.info/documentation/3.6/rspec-core/RSpec/Core/Formatters/Protocol.html